

FastAPI + Docker + PostgreSQL — Setup Notes

These are your own notes explaining everything we've built so far, why each piece exists, and how they all connect. Written assuming you're new to all of this — FastAPI, Docker, and PostgreSQL alike.

1. The big picture

You're building a backend (an API) using **FastAPI** (Python). That API needs somewhere to permanently store data — that's **PostgreSQL**, a database. Instead of installing PostgreSQL directly on Windows, it runs inside **Docker**, in an isolated little box called a "container."

Here's how the three pieces talk to each other:

```
Your browser / Postman
    |
    | HTTP request (e.g. GET /db-check)
    v
+-----+
| FastAPI app      | <-- runs on your Windows machine directly
| (main.py)        |      via: uvicorn main:app --reload
+-----+
    |
    | SQL query, over a network connection to localhost:5432
    v
+-----+
| PostgreSQL       | <-- runs INSIDE a Docker container
| (fastapi_postgres)|      Docker just isolates it; from FastAPI's
+-----+              point of view it's a normal database at
                        localhost:5432
```

Two separate things have to be running at the same time for your app to work:

1. The **Postgres container** (via Docker)

2. The **FastAPI server** (via uvicorn, in your own terminal)

2. Python virtual environment (venv)

What it is: A private, isolated copy of Python just for this project, living in the `venv/` folder. Any package you `pip install` goes in there, not into your system-wide Python. This means different projects can use different (even conflicting) package versions without interfering with each other.

Why it matters: If you ever see `'uvicorn' is not recognized`, it's because the venv isn't "activated" in your current terminal — Windows is looking for `uvicorn` in the system Python, not the project's private one.

How to activate it, from `D:\fastapi-app` :

- Command Prompt (cmd.exe): `venv\Scripts\activate.bat`
- PowerShell: `.\venv\Scripts\Activate.ps1`

You'll know it worked because your prompt gets a `(venv)` prefix, e.g.:

```
(venv) D:\fastapi-app>
```

`requirements.txt` is the list of packages (and exact versions) this project depends on — FastAPI itself, the ASGI server (uvicorn), the database toolkit (SQLAlchemy), the Postgres driver (psycopg2-binary), etc. Anyone (including future-you on a new machine) can recreate your exact environment with:

```
pip install -r requirements.txt
```

3. FastAPI — the backend framework

What it is: A Python framework for building APIs. You write plain Python functions, decorate them with routes (`@app.get("/something")`), and FastAPI turns them into a working HTTP API — plus it auto-generates interactive documentation for free.

`main.py`

```

from fastapi import FastAPI, Depends
from sqlalchemy import text
from sqlalchemy.orm import Session

from database import get_db

app = FastAPI()

@app.get("/health")
def health_check():
    return {"status": "healthy"}

@app.get("/db-check")
def db_check(db: Session = Depends(get_db)):
    db.execute(text("SELECT 1"))
    return {"database": "connected"}

```

- `app = FastAPI()` — creates the application object; everything hangs off this.
- `@app.get("/health")` — defines a **route**: "when someone sends a GET request to `/health`, run this function." This one just proves the server itself is alive.
- `@app.get("/db-check")` — same idea, but this one also talks to the database (`SELECT 1` is a trivial query — if it succeeds, the DB connection is working end-to-end).
- `Depends(get_db)` — FastAPI's **dependency injection**: before running `db_check`, FastAPI calls `get_db()` (from `database.py`) to get a database session, hands it to the function as `db`, and automatically closes it afterward — even if the function raises an error.

Running the server

From `D:\fastapi-app`, with the venv activated:

```
uvicorn main:app --reload
```

- `uvicorn` — the actual server program that runs your FastAPI app and listens for HTTP requests. FastAPI itself doesn't include a server; it needs uvicorn (or similar) to actually serve traffic.
- `main:app` — "in the file `main.py`, use the object named `app`."
- `--reload` — watches your files and restarts the server automatically whenever you save a change. Great for development, never used in production.

Once running, you get:

- <http://127.0.0.1:8000/docs> — an interactive page (Swagger UI) where you can see every route and click "Try it out" to call it directly in the browser. This is the fastest way to explore your own API.
- <http://127.0.0.1:8000/health> and `/db-check` — the routes themselves.

A plain <http://127.0.0.1:8000/> returns 404 — that's expected, since we never defined a route at `/`.

4. Docker — running Postgres in a box

What it is: Docker lets you run software inside lightweight, isolated "containers" instead of installing it directly on your machine. A container packages up an application plus everything it needs to run. Think of it as a disposable mini-computer that starts in seconds.

Why we're using it for Postgres: Instead of installing PostgreSQL natively on Windows (which is fiddly to configure and uninstall cleanly), we run the official `postgres` image in a container. It's easy to start/stop/delete/reset without touching your actual system.

What we had to set up:

- Installed **Docker Desktop** (via `winget install Docker.DockerDesktop`)
- Docker needs virtualization support. Your machine's BIOS already had it enabled, but two Windows features were also required and had to be turned on manually (via an elevated PowerShell):
 - `Microsoft-Windows-Subsystem-Linux` (WSL)
 - `VirtualMachinePlatform`
 - Then a system reboot, plus `wsl --update` and `wsl --set-default-version 2`.

Once Docker Desktop is installed and running (check its tray icon says "Running" / "Engine running"), the `docker` command works from any terminal.

5. PostgreSQL — the database itself

What it is: A relational database — data lives in tables with rows and columns, and you query it with SQL.

`PostgreSQL/docker-compose.yml`

Docker Compose is a way to describe a container's configuration in a file instead of typing a long `docker run ...` command every time.

```
services:
  postgres_db:
    image: postgres:16-alpine
    container_name: fastapi_postgres
    restart: always
    environment:
      POSTGRES_USER: myuser
      POSTGRES_PASSWORD: Denis_Jr
      POSTGRES_DB: mydatabase
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./init:/docker-entrypoint-initdb.d

volumes:
  postgres_data:
```

Line by line:

- `image: postgres:16-alpine` — use the official Postgres version 16 image, "alpine" meaning a small/lightweight Linux base.
- `container_name: fastapi_postgres` — the friendly name Docker gives this container (you'll see it in `docker ps`).
- `restart: always` — if the container crashes or Docker restarts, this container starts back up automatically.
- `environment:` — these three variables configure Postgres on its first startup: the superuser username, password, and the name of the default database to create. **These credentials are what your app uses to connect** — see the `.env` file below, where they must match exactly.
- `ports: "5432:5432"` — maps port 5432 inside the container to port 5432 on your Windows machine (`host:container`). This is what makes `localhost:5432` reachable from your FastAPI app, which runs outside Docker.
- `volumes:` — two different kinds:
 - `postgres_data:/var/lib/postgresql/data` — a **named volume**. This is where Postgres actually stores your data permanently. Even if you delete and recreate the container, this volume (managed by Docker) survives, so your data isn't lost. It's

only deleted if you explicitly run `docker compose down -v` (the `-v` removes volumes too).

- `./init:/docker-entrypoint-initdb.d` — a **bind mount** of your local `PostgreSQL/init/` folder. Postgres automatically runs any `.sql` files it finds there — but **only once**, the very first time it starts against an empty data volume. It will NOT re-run on every restart, and it won't pick up edits automatically like `--reload` does for FastAPI. To test changes to init scripts, you must wipe and recreate:
`docker compose down -v && docker compose up -d .`

PostgreSQL/init/01_init.sql

The one-time starter script — creates a sample `items` table:

```
CREATE TABLE IF NOT EXISTS items (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  created_at TIMESTAMP NOT NULL DEFAULT NOW()  
);
```

This is the table we'll build a real CRUD endpoint against next.

Commands you'll use often

Run these from `D:\fastapi-app\PostgreSQL` :

Command	What it does
<code>docker compose up -d</code>	Start the container in the background
<code>docker compose down</code>	Stop and remove the container (data is kept, in the named volume)
<code>docker compose down -v</code>	Stop, remove the container, and delete all data
<code>docker compose ps</code>	Check if the container is running
<code>docker compose logs -f</code>	Watch Postgres's live logs (useful for debugging)

Always manage this container through `docker compose` , from this folder. Don't use plain `docker run postgres` — we hit exactly this problem once already (see Troubleshooting

below): a stray container with none of the right environment variables set, which crashed on startup.

6. Connecting FastAPI to PostgreSQL

`.env`

```
DATABASE_URL=postgresql://myuser:Denis_Jr@localhost:5432/mydatabase
```

A **connection string** in the form `postgresql://<user>:<password>@<host>:<port>/<database>` — these values must exactly match the `environment:` block in `docker-compose.yml`.

`.env` files hold configuration/secrets that shouldn't be hardcoded into your source code. If you ever set up git for this project, `.env` should be added to `.gitignore` so the password never gets committed.

`database.py`

```
import os
from dotenv import load_dotenv
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base

load_dotenv()

DATABASE_URL = os.getenv("DATABASE_URL")

engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
```

```
finally:  
    db.close()
```

- `load_dotenv()` — reads the `.env` file and makes its values available via `os.getenv(...)`.
- `engine` — SQLAlchemy's connection to the database. It doesn't connect immediately; it's more like a reusable "how to reach the database" object.
- `SessionLocal` — a factory for creating individual conversations ("sessions") with the database. Each API request gets its own session.
- `Base` — the class your future table models (Python classes that map to SQL tables) will inherit from. Not used yet, but this is the foundation for when we define models like `class Item(Base): ...`.
- `get_db()` — a generator function used as a FastAPI dependency. It opens a session, `yield`s it to whichever endpoint asked for it, and the `finally: db.close()` guarantees the session is cleaned up afterward no matter what happens in the endpoint.

SQLAlchemy is the "ORM" (Object-Relational Mapper) — it lets you work with the database using Python objects and method calls instead of writing raw SQL strings everywhere (though you still can, like the `SELECT 1` in `/db-check`).

7. Everyday workflow — starting everything up

Two terminals, two things running:

Terminal 1 — the database:

```
cd D:\fastapi-app\PostgreSQL  
docker compose up -d
```

Terminal 2 — the API:

```
cd D:\fastapi-app  
.\venv\Scripts\Activate.ps1    # (or activate.bat in cmd.exe)  
uvicorn main:app --reload
```

Then test in your browser or Postman:

- `GET http://127.0.0.1:8000/health` → `{"status": "healthy"}`

- `GET http://127.0.0.1:8000/db-check` → `{"database": "connected"}`
- `http://127.0.0.1:8000/docs` → interactive API explorer

8. Troubleshooting log (things we already ran into)

- **404 Not Found on GET /** — Not a bug. We never defined a route at `/`, only `/health` and `/db-check`. Expected behavior.
- **docker not recognized** — Docker Desktop wasn't installed yet. Installed via winget.
- **Docker said "virtualization not enabled, contact your administrator"** — Misleading generic message. BIOS virtualization was actually already on; the real issue was the Windows features (WSL + Virtual Machine Platform) not being enabled yet. Turned on via an elevated PowerShell, then rebooted.
- **uvicorn not recognized** — The venv wasn't activated in that terminal (no `(venv)` prefix in the prompt), so Windows couldn't find it. Also, running it from the wrong folder (`PostgreSQL/` instead of the project root) wouldn't find `main.py` anyway.
- **Database is uninitialized and superuser password is not specified** — Turned out to be a second, unrelated container named `postgres1` that had been started with a plain `docker run` (no environment variables set), separate from our `docker-compose.yml` - managed `fastapi_postgres` container. Removed the stray one; our real container was fine the whole time.

9. Glossary

Term	Meaning
venv	An isolated set of Python packages just for this project
FastAPI	The Python framework used to define API routes
uvicorn	The server program that actually runs the FastAPI app and handles HTTP traffic
route / endpoint	A specific URL path + HTTP method your API responds to, e.g. <code>GET /health</code>
Docker	Runs applications in isolated "containers"
image	A blueprint for a container (e.g. <code>postgres:16-alpine</code>)

Term	Meaning
container	A running instance of an image
Docker Compose	A tool/file format for defining and running multi-container setups declaratively
volume (named)	Docker-managed persistent storage that survives container recreation
bind mount	A folder on your machine mapped directly into the container
PostgreSQL / Postgres	The relational database itself
SQLAlchemy	Python's ORM — lets you interact with the database using Python objects
psycopg2	The low-level driver SQLAlchemy uses to actually speak to Postgres
ORM	Object-Relational Mapper — maps database tables to Python classes
dependency injection (<code>Depends</code>)	FastAPI's way of supplying things (like a DB session) into your route functions automatically
<code>.env</code> file	Stores config/secrets outside your source code
connection string	A single string encoding how to connect to a database (user, password, host, port, db name)

10. What's next

1. Write a proper SQLAlchemy **model** for the `items` table (a Python class mapped to that table).
2. Build a real **CRUD endpoint** — create and read items via the API, backed by the database.
3. Test everything with **Postman** — including what happens when you send bad data, and what HTTP status codes come back and why (`200` , `201` , `404` , `422` , `500` , etc).

4. Later: introduce **Alembic** for database migrations — the proper way to evolve your schema over time without wiping data (unlike our current init-script approach, which only runs once).